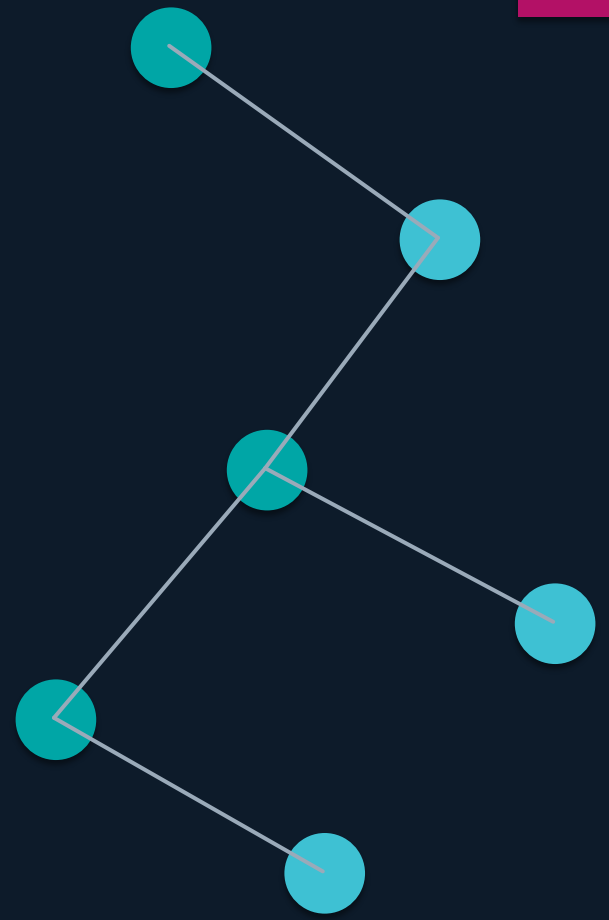
REKAYASA PERANGKAT LUNAK

Software Engineering

PERTEMUAN 1 • PENDAHULUAN



Manorang Shotang,M.Kom.,Ph.D

Silabus & Pustaka

Pertemuan 1 • Pengenalan Rekayasa Perangkat Lunak

1. Pengenalan proyek perangkat lunak
2. Perencanaan proyek perangkat lunak
3. Konsep & prinsip analisis
4. Pemodelan analisis
5. Prinsip & konsep desain
6. Metode desain
7. Teknik pengujian perangkat lunak
8. Strategi pengujian perangkat lunak
9. Pemeliharaan perangkat lunak

Referensi / Pustaka

1 **Roger S. Pressman**

Software Engineering: A Practitioner’s Approach, 5th ed., McGraw-Hill

2 **Tavri D. Mahyuzir**

Pengantar Rekayasa Perangkat Lunak, Elex Media Komputindo

3 **Ian Sommerville**

Software Engineering, 7th ed., Addison-Wesley

Fokus Utama

01

Rekayasa Perangkat Lunak (RPL)

RPL berfokus pada pengembangan perangkat lunak secara terencana, sistematis, terukur, dan berorientasi kualitas.

01 Terencana

Setiap tahap memiliki tujuan, aktivitas, dan keluaran yang jelas.

02 Sistematis

Proses pengembangan mengikuti metode, standar, dan praktik yang terstruktur.

03 Berkualitas

Hasil akhir memenuhi kebutuhan pengguna, andal, efisien, dan mudah dipelihara.

Mulai dari kebutuhan pengguna → proses pengembangan → perangkat lunak yang siap digunakan.

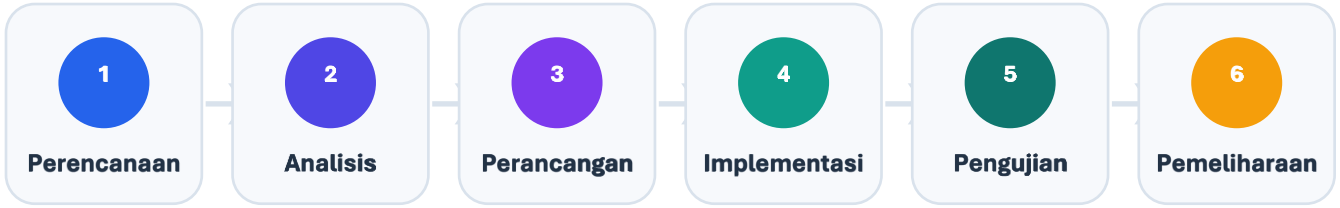
Ruang Lingkup RPL

SDLC dan Manajemen Proyek

Dua fondasi utama untuk menjaga proses pengembangan tetap terarah dan terkendali.

SDLC — Software Development Life Cycle

Mengatur tahapan pengembangan dari awal hingga perangkat lunak dipelihara.



Manajemen Proyek

Menjaga proyek tetap sesuai target melalui pengelolaan:

- **Waktu**
- Biaya
- Sumber daya
- Risiko
- Pembagian tugas

Tujuan: proyek selesai sesuai ruang lingkup, waktu, dan sumber daya yang direncanakan.

Tahap Pengembangan

Dari kebutuhan menuju solusi

Tahapan ini menerjemahkan kebutuhan pengguna menjadi rancangan dan produk perangkat lunak.

03

Analisis Kebutuhan

Apa yang dibutuhkan?

Mengidentifikasi kebutuhan pengguna dan menerjemahkannya menjadi spesifikasi sistem yang jelas.

04

Perancangan Sistem

Bagaimana solusinya?

Merancang arsitektur, basis data, antarmuka, serta komponen perangkat lunak sebelum implementasi.

05

Implementasi

Mewujudkan rancangan

Menerjemahkan desain menjadi kode program menggunakan bahasa dan teknologi yang sesuai.

Kebutuhan → Spesifikasi → Rancangan → Kode

Kualitas dan Keberlanjutan

Pengujian, Pemeliharaan, dan Tujuan Akhir

RPL memastikan perangkat lunak tidak berhenti pada tahap “selesai dibuat”, tetapi terus memberi nilai saat digunakan.

06

Pengujian

Memastikan perangkat lunak berfungsi sesuai kebutuhan serta menemukan dan memperbaiki kesalahan.

07

Pemeliharaan

Melakukan perbaikan, pembaruan, pengembangan fitur, dan penyesuaian terhadap perubahan kebutuhan.

TUJUAN AKHIR RPL

Perangkat lunak yang berkualitas, sesuai kebutuhan pengguna, efisien, tepat waktu, dan mudah dipelihara.

Intinya: RPL menggabungkan proses, manusia, teknologi, dan pengelolaan proyek untuk menghasilkan solusi perangkat lunak yang bernilai.

Tujuan Pembelajaran

01

Memahami fondasi Software Engineering

Mahasiswa mengenali perangkat lunak, proses pengembangan, model proses, serta tanggung jawab etika dan profe

A **Konsep**

Apa itu software & software engineering

B **Proses**

Bagaimana software dikembangkan

C **Profesional**

Kualitas, etika, keamanan & dampak

Definisi Perangkat Lunak

Software mencakup

Instruction: program komputer yang menghasilkan fungsi & kinerja yang diinginkan.

Data structure: struktur data yang memungkinkan program memanipulasi informasi.

Documents: dokumentasi yang menjelaskan operasi & penggunaan program.

Dalam praktik

Program komputer + dokumentasi terkait: kebutuhan, rancangan, dan user manual.

Produk dapat dibuat untuk pengguna umum maupun pengguna khusus.

G Generic

Dibangun untuk dijual kepada banyak pengguna. Contoh: aplikasi perkantoran seperti spreadsheet atau pengolah kata.

C Bespoke / Custom

Dibangun khusus untuk pengguna atau pemesan sesuai kebutuhan.

Penggunaan Software

1. **Mengendalikan komputer**

   Contoh: sistem operasi

2. **Mengomunikasikan informasi**

   Contoh: software jaringan

3. **Membuat & mengendalikan program lain**

   Tool dan lingkungan pengembangan

Perkembangan Pemanfaatan Software

- Menjadi faktor penting dalam pengambilan keputusan bisnis.
- Menjadi landasan investigasi ilmiah dan pemecahan masalah.
- Menjadi pembeda penting antara produk/layanan modern dan tradisional.
- Terintegrasi dalam berbagai sistem: telekomunikasi, transportasi, militer, medis, industri, dan lainnya.

Jenis Perangkat Lunak

Bagian 1 • Klasifikasi utama

1 System Software

Mengatur software lain dan banyak berinteraksi dengan hardware.

2 Real-time Software

Memiliki waktu tanggap/response time yang singkat.

3 Business Software

Mendukung proses seperti penggajian, penjualan, dan persediaan; banyak berhubungan dengan data.

4 Engineering & Scientific

Banyak memproses angka; contoh peramalan cuaca dan aplikasi geografis.

Jenis Perangkat Lunak

Bagian 2 • Perangkat yang dekat dengan pengguna

1 Embedded Software

Tersimpan/tertanam pada perangkat dan mengatur perangkat keras. Contoh: mesin cuci, microwave, kulkas.

2 Personal Computer Software

Spreadsheet, pengolah kata, dan multimedia.

3 Web-based Software

Berbasis teknologi web; contoh e-commerce.

4 Artificial Intelligence Software

Memanfaatkan teknik AI; contoh pengenalan wajah.

Rekayasa Perangkat Lunak

Software Engineering

Disiplin yang berfokus pada seluruh aspek pembuatan software, mulai dari spesifikasi hingga pemeliharaan setelah software digunakan.

1 **Sistematis**

Mengikuti pendekatan yang terstruktur dan teratur.

2 **Teknik & alat**

Menggunakan metode dan tools yang sesuai dengan masalah.

3 **Kendala nyata**

Mempertimbangkan batasan pembangunan dan sumber daya yang tersedia.

Latar Belakang Software Engineering

- Ketergantungan manusia, masyarakat, dan industri terhadap software semakin besar.
- Dibutuhkan proses produksi software yang cepat, andal, dan ekonomis.
- Semakin banyak sistem dikendalikan oleh perangkat lunak.
- Software Engineering menyediakan teori, metode, dan alat untuk pembangunan software secara profesional.
- Fokus pada efektivitas biaya sepanjang pembangunan dan evolusi software.

Tantangan Software Engineering

1 Heterogenitas

Keragaman platform, perangkat, dan lingkungan meningkat.

2 Keragaman sistem

Tipe sistem software semakin beragam.

3 Metode & tools

Teknik dan alat bantu harus dipilih sesuai konteks.

4 Perubahan

Bisnis dan kebutuhan sosial terus berubah.

5 Ketepatan waktu

Software perlu dikembangkan dan dirilis tepat waktu.

6 Keamanan & kepercayaan

Software harus aman dan dapat dipercaya.

CASE • Computer-Aided Software Engineering

Apa itu CASE?

Software aplikasi yang digunakan untuk mendukung proses pembuatan software.

1 **Analisis kebutuhan**

Didukung oleh CASE tools

2 **Pemodelan sistem**

Didukung oleh CASE tools

3 **Debugging**

Didukung oleh CASE tools

4 **Testing**

Didukung oleh CASE tools

Karakteristik Software yang Baik

1. Usability — mudah digunakan
2. Performance — kinerja sesuai kebutuhan
3. Reliability — dapat diandalkan
4. Maintainability — mudah dirawat/diperbaiki
5. Efficiency — menggunakan sumber daya secara efektif
6. Attractive UI — antarmuka menarik
7. Long lifetime — memiliki siklus hidup yang memadai

Jenis Aplikasi PL (1)

1

Perangkat lunak sistem

Melayani program-program lain. Contoh: sistem operasi, driver, kompilator, interpreter, utility.

2

Perangkat lunak waktu nyata

Memonitor, menganalisis, mengontrol, dan melaporkan kejadian dunia nyata dengan respons cepat.

Jenis Aplikasi PL (2)

1

Perangkat lunak teknik & ilmu pengetahuan

Menangani bidang teknik dan ilmu pengetahuan secara rinci. Contoh: simulasi, astronomi, vulkanologi, otomasi pabrik.

2

Embedded system

Software yang ditempelkan/dilekatkan pada perangkat lain. Contoh: kamera digital, GPS, automobil, microwave, kulkas cerdas.

Jenis Aplikasi PL (3)

1

Data processing

Mengolah data untuk menghasilkan informasi atau keputusan tertentu. Contoh: billing telepon, pengolah statistik.

2

Information system

Menyediakan informasi dari suatu sistem secara lebih detail. Contoh: website, perpustakaan digital.

Jenis Aplikasi PL (4)

1

Perangkat lunak sensor

Mengukur dan mengatur keadaan khusus; dapat termasuk embedded system. Contoh: pengatur suhu.

2

Communication software

Menghubungkan atau mengomunikasikan objek satu dengan lainnya. Contoh: router, telepon seluler.

Jenis Aplikasi PL (5)

1

Perangkat lunak kantor

Membantu tugas perkantoran. Contoh: word processing, spreadsheet, video conference.

2

Perangkat lunak grafis

Mendukung perancangan grafis. Contoh: pembuatan film dan poster.

Jenis Aplikasi PL (6)

AI

Perangkat lunak kecerdasan

Menggunakan algoritma non-numeris untuk masalah kompleks. Contoh: sistem pakar, pembuktian teorema, game strategi, jaringan saraf tiruan.

Tanggung Jawab Etika & Profesi

1 **Konfidensialitas**

Menjaga kerahasiaan informasi.

2 **Kompetensi**

Bekerja sesuai kemampuan dan standar profesional.

3 **HAKI**

Menghormati hak kekayaan intelektual.

4 **Penyalahgunaan**

Mencegah penggunaan komputer/software secara tidak semestinya.

Software Process

Definisi

Serangkaian kegiatan yang dilakukan untuk menghasilkan sebuah produk software.

P

Plan-driven process

Semua proses dan aktivitas direncanakan di awal, kemudian kemajuan pekerjaan diukur terhadap rencana tersebut.

Aktivitas Utama Software Process

1 **Specification**

Mendefinisikan fungsi software dan batasan pengoperasiannya.

2 **Design & Implementa**

Mendesain dan membuat software sesuai spesifikasi.

3 **Validation**

Memvalidasi agar software sesuai kebutuhan/customer.

4 **Evolution**

Mengembangkan software untuk memenuhi perubahan kebutuhan/customer.

Software Process Model

Definisi

Representasi yang disederhanakan dari suatu proses software; menjelaskan proses dari sudut pandang tertentu.

1 Workflow

Aktivitas yang berurutan

2 Output

Produk/hasil

3 Roles

Siapa melakukan apa

4 Conditions

Pre-condition & post-condition

Keluarga Software Process Model

1 **Sequential**

Waterfall • V-Model

2 **Iterative / Incremental**

Spiral • RAD • RUP

3 **Agile**

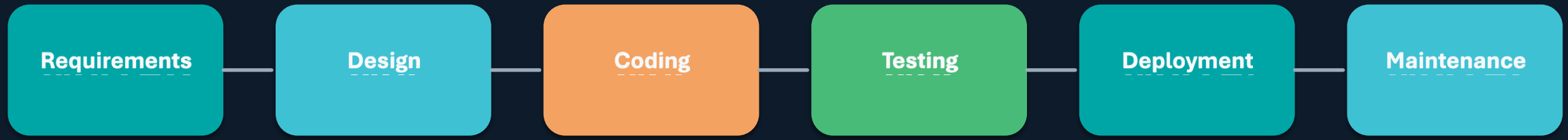
Extreme Programming • Scrum

4 **Reuse-oriented**

Integrasi komponen yang sudah ada

WATERFALL

Model sekuensial



Inti model: setiap fase diselesaikan sebelum beralih ke fase berikutnya.

WATERFALL

Tahapan dan konteks penggunaan

Tahapan

1. Requirements
2. Design
3. Coding
4. Testing
5. Deployment
6. Maintenance

Cocok digunakan ketika

Scope & requirement sudah diketahui dengan jelas sejak awal.

Proyek mengharuskan dokumentasi yang lengkap.

Sistem besar dikembangkan oleh beberapa tim/lokasi.

Pro–Cons Waterfall

PRO

Disiplin proses kuat.

Setiap fase memiliki titik awal & akhir yang jelas.

CONS

Rigid/kaku.

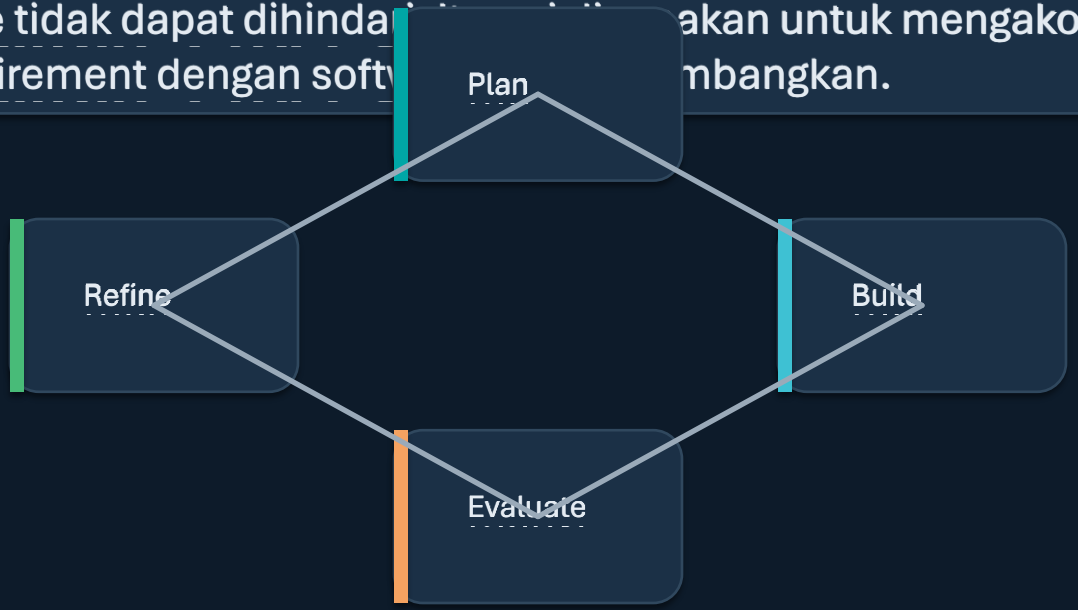
Delivery relatif lebih lama.

Dokumentasi lebih banyak.

Proses Iterasi

Mengapa iterasi?

Perubahan software tidak dapat dihinda akan untuk mengakomodasi perubahan dan menyesuaikan requirement dengan soft mbangkan.



Pro–Cons Iterasi / Incremental

PRO

Biaya akibat perubahan requirement dapat ditekan.

Feedback customer lebih mudah diperoleh.

Delivery hasil pekerjaan lebih cepat.

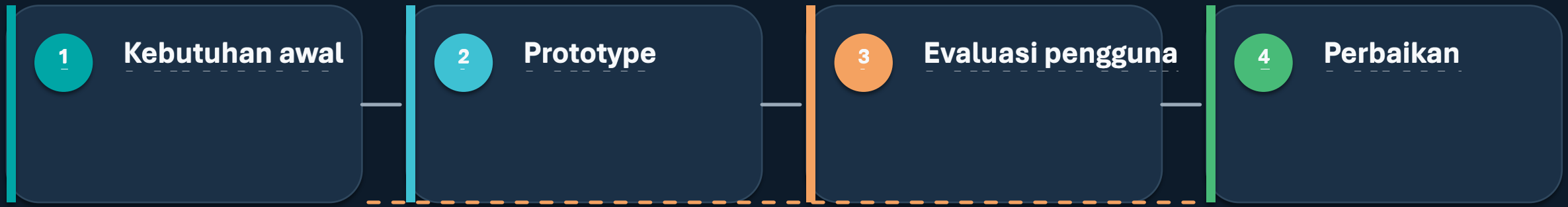
CONS

Proses dapat kurang terdokumentasi.

Perubahan yang terlalu sering dapat memengaruhi struktur software.

PROTOTYPING

Membuat representasi awal untuk mendapatkan umpan balik



SPIRAL

Iteratif dengan penekanan pada pengelolaan risiko

1. Tujuan
2. Analisis risiko
3. Pengembangan
4. Evaluasi



Agile Methodology

Prinsip inti

1. Individu & interaksi > proses & tools
2. Software yang bekerja > dokumentasi komprehensif
3. Kolaborasi customer > negosiasi kontrak
4. Respons terhadap perubahan > mengikuti rencana

Contoh

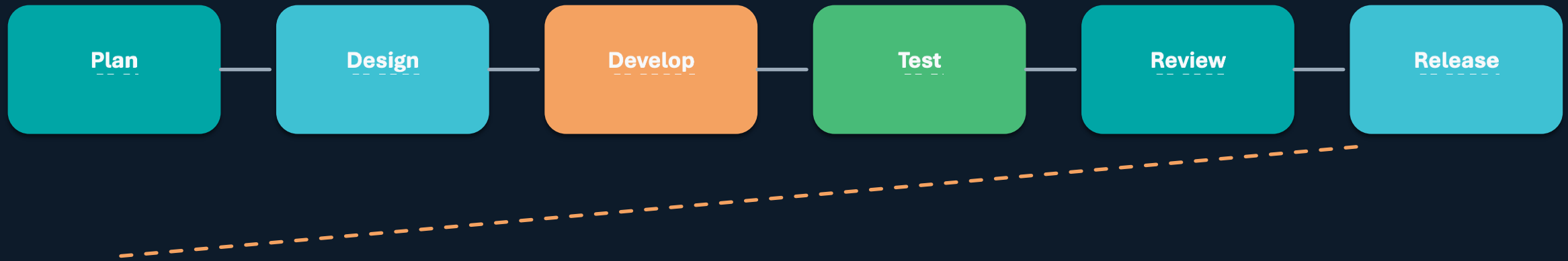
Pair Programming

Contoh

Scrum

Agile Development Life Cycle

Siklus pendek, berulang, dan berbasis feedback



Setiap iterasi menghasilkan bagian software yang dapat dievaluasi dan menjadi masukan untuk siklus berikutnya.

Pro–Cons Agile Methodology

PRO

Delivery lebih cepat (mingguan, bukan bulanan).

Lebih fleksibel daripada waterfall.

Responsif terhadap perubahan scope.

Meningkatkan akurasi estimasi.

Mengurangi feature creep.

Catatan penting

Keberhasilan membutuhkan keterlibatan project owner/customer yang intensif.

Kesalahpahaman tentang agile dapat menghasilkan budaya kerja yang buruk.

Fokus pada potongan delivery kecil dapat menyulitkan pemikiran strategis bila tidak dikelola.

Reuse-Oriented Model

Gagasan utama

Sistem dibangun dengan mengintegrasikan komponen-komponen sistem yang sudah ada.

1. Analisis kompo
2. Modifikasi requ
3. Desain dengan
4. Pengembangan
5. Validasi sistem

Reuse-Oriented Model

Tipe komponen software

W Web service

Komponen yang menyediakan fungsi melalui layanan yang dapat diintegrasikan oleh sistem lain.

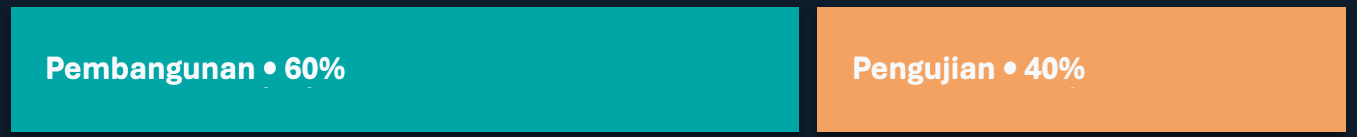
F Component-based framework

Framework berbasis komponen, misalnya .NET atau J2EE.

Biaya Rekayasa Perangkat Lunak

Gambaran distribusi biaya

Secara kasar, materi sumber menyebut 60% untuk pembangunan dan 40% untuk pengujian. Pada software custom, biaya evolusi dapat melebihi biaya pembangunan.



Faktor yang memengaruhi

Tipe sistem.

Kinerja yang dibutuhkan.

Keandalan sistem.

Model pembangunan yang digunakan.

Catatan: distribusi biaya bergantung pada konteks proyek; angka di atas mengikuti materi sumber.

Penyebab Permasalahan Etika dalam Sistem Informasi

1

Pengaruh TI

Pengaruh teknologi informasi yang mendalam membawa konsekuensi terhadap kehidupan manusia dan memunculkan pertanyaan etika.

2

Tanggung jawab manajer

Manajer yang menentukan penerapan TI dalam organisasi ikut bertanggung jawab terhadap permasalahan etika yang muncul.

Permasalahan Etika dalam Lingkungan Sistem Informasi

1. Privacy
2. Intellectual Property Rights
3. Penghentian kerja
4. Security
5. Accuracy & Health

Privacy

Tuntutan seseorang untuk tidak dicampuri, diawasi, atau diganggu oleh orang lain, organisasi, maupun negara. Perlindungan privacy diatur melalui hukum di berbagai negara.

Intellectual Property Rights

Isu utama

Pembajakan software melanggar hak kekayaan intelektual.

- Mudah dilakukan dan dapat dilakukan di berbagai tempat.
- Hasil salinan tampak sama dengan produk yang dibeli.
- Harga software asli dapat dianggap mahal.
- Anggapan bahwa perusahaan pembuat software tidak dirugikan tidak menghilangkan pelanggaran HAKI.

Penghentian Kerja & Security

Dampak penerapan TI terhadap pekerjaan

Positif: meningkatkan produktivitas.

Positif: meningkatkan kualitas pekerjaan.

Positif: memperkaya pekerjaan.

Potensi negatif: sebagian pekerjaan manusia dapat digantikan oleh teknologi demi efisiensi.

Security

Kelalaian maupun kesengajaan dapat merusak keamanan sistem.

Contoh risiko: meninggalkan terminal tanpa dijaga.

Menuliskan password di tempat yang dapat dibaca orang lain.

Memberikan password kepada orang lain.

Accuracy & Health

Accuracy

Software yang dipasarkan seharusnya memiliki kualitas yang memadai; menjual sistem yang masih mengandung kesalahan (bug) menimbulkan persoalan etika.

1

Repetitive Stress Injury (RSI)

Aktivitas komputer yang berulang dan postur yang kurang tepat dapat memicu keluhan fisik. Materi menekankan pentingnya rancangan kerja yang ergonomis.

Health: Computer Vision Syndrome & Technostress

2 Computer Vision Syndrome

Materi mengaitkannya dengan penggunaan monitor yang berkepanjangan, seperti mata kering/iritasi, pandangan kabur, dan pusing. Pencegahan perlu memperhatikan kebiasaan penggunaan layar dan ergonomi.

3 Technostress

Tekanan/stres yang berkaitan dengan penggunaan komputer secara terus-menerus dapat memengaruhi kenyamanan dan perilaku kerja.

Pesan penutup: software yang baik bukan hanya berfungsi, tetapi juga aman, dapat dipelihara, dan dikembangkan secara bertanggung jawab.